SettleUpExpenses

High-Precision Group Expense Sharing, Algorithmic Debt Minimization & Role-Enforced Financial Settlement Platform

Architecture: React 18 SPA + Node.js Express + PostgreSQL | Precision: Integer-Paise Math (Zero Floating Loss) | Optimization: Greedy Min-Cash-Flow O(N-1) | Payments: NPCI UPI Deep Link & Dynamic QR | Testing: 133 Automated Tests Passing

01 Executive Project Summary

SettleUpExpenses is a production-ready, full-stack financial collaboration platform designed to eliminate the social friction, mathematical errors, and payment tracking complexities inherent in shared group living, travel, and events. Unlike generic split-sheet calculators, SettleUpExpenses operates as an institutional-grade financial clearinghouse featuring **6 mathematical split modalities**, **multi-payer support**, **deterministic integer-paise accounting**, **greedy debt minimization**, and a **role-enforced settlement verification state machine** integrated with India's **Unified Payments Interface (UPI)**.

02 The Core Problem Statement

Problem In A Nutshell

In modern communal settings (roommates in shared apartments, group excursions, dining out, and project teams), expenditures are frequent, uneven, and shared among overlapping subgroups. Existing tracking solutions (spreadsheets, basic calculators, and legacy apps) suffer from six fundamental architectural flaws that lead to fractional penny leakage, transaction paralysis, and social disputes.

The 6 Inherent Sub-Problems Addressed by SettleUpExpenses

1. Floating-Point Rounding & Penny Loss

ARITHMETIC FLAW

Computers cannot represent decimal fractions (like `1/3`) precisely in IEEE 754 floating-point math. Splitting ₹100 among 3 people naively produces ₹33.33 each, leaving ₹0.01 lost in limbo. Across hundreds of expenses, this causes cumulative ledger drift where the sum of group balances fails to equal zero.

2. The Multi-Payer Expense Dilemma

STRUCTURAL FLAW

Real-world bills are rarely paid by a single individual. For a ₹5,000 villa booking, Member A might pay ₹3,000 and Member B pays ₹2,000 for a group of 5. Standard apps force artificial workarounds (creating multiple phantom expenses), leading to confusing ledgers and double-counting errors.

3. Rigid, Unrealistic Split Modalities

FLEXIBILITY FLAW

Real life requires complex divisions: percentage-based splits, exact rupee quotas, family shares (weights), custom adjustments (e.g., non-drinkers paying less), and itemized café receipts with proportional tax/tip allocations. Without dedicated math engines, users are forced to calculate shares manually on paper.

4. Circular Debt & Transaction Explosion

COMPLEXITY FLAW

In a group of N people, pairwise debts naturally grow at `O(N²)` complexity (e.g. 10 people can have 45 separate bilateral debts). Member A owes B ₹500, B owes C ₹500, and C owes A ₹500. This creates an absurd web of circular bank transfers, unnecessary payment fees, and tracking fatigue.

5. Settlement Fraud & Asymmetric Trust

SECURITY FLAW

In typical group apps, any member can click "Settle" on anyone's debt, or a debtor can falsely mark their debt as paid without the creditor receiving money. This lack of role-based verification causes premature balance resets and dispute-ridden group dynamics.

6. Disconnect Between Ledger and Rails

UX FRICTION

Users must switch between the expense tracking app and their banking/UPI app (Google Pay, PhonePe, Paytm). They must manually ask for the recipient's UPI ID or phone number, copy-paste it, and type the exact amount—frequently leading to typo errors, incorrect payees, and unverified transfers.

03 The Engineered Solution

SettleUpExpenses addresses each of these foundational challenges through a mathematically rigorous, verified, and secure application architecture:

The 5 Pillars of SettleUpExpenses Architecture

1. **Deterministic Integer-Paise Math:** All monetary values are internally stored, computed, and transferred strictly in whole integer Paise (1 INR = 100 Paise). Fractional remainders are systematically distributed deterministically to ensure the Zero-Sum Invariant (`∑ NetBalances = 0`) is maintained at all times.
2. **Greedy Min-Cash-Flow Optimization:** A graph reduction algorithm partitions all members into net debtors and net creditors, matching the maximum debtor with the maximum creditor greedily. This reduces `O(N²)` pairwise obligations down to at most `N - 1` simplified settlement transactions.
3. **Role-Enforced Two-Step Verification:** Only the debtor can initiate a payment proposal (which enters `PENDING` status). Non-debtors cannot settle someone else's debt. Only the creditor (receiver) possesses authorization to `Confirm` or `Reject` the payment. Balances update strictly upon confirmation.
4. **Native NPCI UPI Integration:** Dynamic generation of standard `upi://pay` deep links and high-definition QR codes pre-filled with recipient VPA, verified name, exact INR amount, and audit note for instant mobile execution.
5. **Immutable Audit Ledger & Concurrency Safety:** Complete historical event log across expenses, void events, and settlement transitions, backed by database transactions with `SELECT ... FOR UPDATE` concurrency locks and idempotency keys.

04 Detailed Implementation Inventory (What Is Built Till Now)

The following modules are fully engineered, unit-tested, integration-tested, and active in the live repository:

System Module	Features & Technical Implementation	Status & Verification
1. User & Authentication	• JWT Token-based authentication with Bearer headers. • Passwords hashed with `bcryptjs` (salted 10 rounds). • Profile management with UPI VPA validation and storage. • User registration, login, and secure session persistence.	**COMPLETED & TESTED** `authController.js` 4 passing unit & API tests
2. Group Collaboration & Governance	• Group creation with unique 8-character alphanumeric invite codes. • Direct join-by-invite-code engine. • Role hierarchy: Group Creator / Owner vs Active Members. • **Safety Guard:** Members are strictly blocked from leaving a group if their net balance is non-zero (must settle all debts first).	**COMPLETED & TESTED** `groupController.js` Verified with active PostgreSQL DB

3. Multi-Payer Expense Engine	• Supports single or multiple payers contributing different amounts.
	• **6 Mathematical Split Modalities:**
	1. *Equal Split:* Deterministic remainder allocation.
	2. *Exact Amount Split:* User-specified rupee allocations.
	3. *Percentage Split:* Proportional % validation (sum = 100%).
	4. *Shares Split:* Proportional weights allocation.
	5. *Adjustment Split:* Base equal split with +/- offsets.
	6. *Itemized Receipt:* Item prices + proportional shared tax/tip.
	• Soft-delete / Void expense engine with audit record preservation.
		COMPLETED & TESTED
		`expenseController.js`
		`src/utils/splits.js`
		All 6 methods verified (36 tests)

4. Financial Math & Debt Graph	• `calculateDetailedDebts`: Deterministic proportional payer allocation generating exact pairwise debts with bilateral netting.
	• `calculateBalancesAndOptimization`: Greedy Min-Cash-Flow engine minimizing total transfers to at most N - 1.
	• Dual-view toggle on UI: *Optimized Settlement Plan* vs *Detailed Person-to-Person Debts*.
		COMPLETED & TESTED
		`src/utils/optimizer.js`
		`src/utils/debts.js`
		Monte Carlo property verified

5. Role-Enforced Settlement Flow	• **Role Authorization:** Only the debtor can record/settle their own debt. Non-debtors have buttons hidden/disabled.
	• **Status Transitions:** Payment record initiates state as `PENDING`.
	• **Receiver Authority:** Only the recipient can `Confirm` or `Reject`.
	• **Balance Invariant:** Balances remain 100% unchanged during `PENDING` or `REJECTED` states; net balances update strictly on `CONFIRMED`.
		COMPLETED & TESTED
		`settlementController.js`
		`balanceUpdateTiming.test.js`
		Concurrency-safe locks

System Module	Features & Technical Implementation	Status & Verification

6. UPI Integration & Mobile Pay	• NPCI-compliant `upi://pay` URI generation.
	• Dynamic QR code generator modal with 1-click payment link.
	• Pre-fills recipient VPA, recipient name, exact amount, and reference note.
		COMPLETED & TESTED
		`src/utils/upi.js`
		`UpiQrModal.jsx`

7. Reliability & Idempotency	• Row-level concurrency locking using `SELECT ... FOR UPDATE`. • `Idempotency-Key` header support preventing double submissions. • 133 automated unit, integration, and randomized stress tests.	**COMPLETED & TESTED** 133 / 133 tests passing

05 Mathematical Invariants & Algorithmic Foundations

1. The Zero-Sum Conservation Invariant

In any closed financial pool, money is neither created nor destroyed. The total sum of all members' net balances must equal zero paise at every moment in time:

```
∑_{i=1}^{M} NetBalance_i = ∑_{i=1}^{M} (TotalPaid_i - TotalShare_i + SettledReceived_i - SettledPaid_i) ≡ 0 (paise)
```

2. Greedy Min-Cash-Flow Debt Simplification Algorithm

To resolve the `O(N²)` circular debt problem, SettleUpExpenses extracts net balances from all expenses and confirmed settlements, splits members into two priority heaps, and greedily matches the maximum debtor with the maximum creditor:

```
1. Compute Net Balance for each member: Net[u] = Total_Paid[u] - Total_Owed[u]
2. Partition into Debtors (Net < 0) and Creditors (Net > 0)
3. While Debtors and Creditors are non-empty:
    a. Pick max debtor D with debt |Net[D]| and max creditor C with credit Net[C]
    b. Transfer amount T = min(|Net[D]|, Net[C])
    c. Record suggested transaction: "D pays C amount T"
    d. Update Net[D] += T, Net[C] -= T; remove settled members
4. Result: Guaranteed exact settlement in at most N - 1 total transactions.
```

3. The Deterministic Remainder Allocation Invariant

When dividing an expense of `A` paise among `K` participants:

```
BaseShare = floor(A / K)
Remainder = A mod K
For the first (Remainder) participants in sorted user order: Share = BaseShare + 1
For the remaining participants: Share = BaseShare
Guarantee: ∑_{i=1}^{K} Share_i = BaseShare × K + Remainder ≡ A (Zero penny loss)
```

06 Role-Based Settlement State Machine

To prevent fraud and maintain financial integrity, settlements strictly follow a finite state machine:

```
[DEBT OCCURS] → Debtor owes Creditor in Group
  ↓
[RECORD PAYMENT] → Executed strictly by Payer (Debtor) → Status: PENDING (Balances UNCHANGED)
  ↓
  ├--- [REJECT] → Executed strictly by Receiver (Creditor) → Status: REJECTED (Cancelled, Balances UNCHANGED)
  ↓
[CONFIRM] → Executed strictly by Receiver (Creditor) → Status: CONFIRMED
  ↓
[LEDGER UPDATE] → Payer net balance +Amount, Receiver net balance -Amount → Optimizer recalculates
```

07 Database Architecture & Entity Relationships

Table Name	Core Fields & Foreign Keys	Responsibility
`users`	`id, email, password_hash, name, upi_id, created_at`	User identity, authentication credentials, and default UPI VPA.
`groups`	`id, name, description, created_by, invite_code, created_at`	Collaborative expense pools with unique invite code access.
`group_members`	`id, group_id, user_id, role, status, joined_at`	Membership tracking, owner vs member roles, active states.
`expenses`	`id, group_id, created_by, description, total_amount_paise, split_type, is_voided, voided_by`	Main expense record with audit tracking and soft-void flags.
`expense_payers`	`id, expense_id, user_id, amount_paise`	Multi-payer contributions mapping who paid what portion.
`expense_participants`	`id, expense_id, user_id, share_amount_paise`	Individual liabilities per participant for the expense.
`settlements`	`id, group_id, from_user, to_user, amount_paise, status, payment_method, confirmed_by, confirmed_at`	Settlement lifecycle records with role-enforced approval tracking.
`activity_logs`	`id, group_id, actor_id, entity_type, entity_id, action, metadata, created_at`	Immutable chronological audit log for group financial history.

08 Technology Stack & Architecture Details

Frontend Layer

- **React 18:** Modern component-driven single-page architecture.
- **Vite 8:** High-performance bundler and HMR runtime.
- **Lucide Icons:** Consistent, clean visual iconography.
- **Custom Design System:** Lightweight, bespoke CSS variables; responsive on mobile & desktop.
- **QR Code Engine:** In-browser generation of high-resolution UPI QR codes.

Backend API Layer

- **Node.js & Express 5:** RESTful API server with asynchronous controllers.
- **JWT & bcryptjs:** Stateless authentication & secure salted hashing.
- **Transaction Control:** Atomic multi-table updates using PostgreSQL transactions.
- **Idempotency Layer:** Prevents duplicate transactions using custom idempotency headers.
- **Security Middlewares:** Role verification, rate limiting, and input sanitization.

Database & Testing

- **PostgreSQL:** ACID-compliant relational DB with foreign keys and check constraints.
- **node-postgres (pg):** Connection pooling with parameterized SQL queries.
- **Node Test Runner:** Native test framework (`node --test`).
- **Supertest:** End-to-end REST API integration testing.
- **Monte Carlo Testing:** 100 randomized multi-user stress tests validating conservation.

09 How to Pitch & Understand This Project

If Explaining to an Evaluator, Recruiter, or Colleague:

"SettleUpExpenses is not just a digital ledger; it is an intelligent financial settlement system. In real-world expense sharing, standard calculators break down because of floating-point penny discrepancies, multi-payer contributions, complex split ratios, and circular debt chaos where 5 friends make 10 unnecessary bank transfers."

"SettleUpExpenses solves this with mathematical rigor: storing all currency in integer paise with zero precision loss, supporting 6 sophisticated split models (including itemized tax-apportioned bills), and applying a Greedy Min-Cash-Flow graph algorithm that simplifies complex debt webs into the minimum possible transactions."

"On top of that, it enforces a role-based approval protocol: debtors record payments as pending, creditors must verify and confirm receipts before balances ever change, and native UPI QR links allow instant 1-click bank transfers without typing recipient details."

10 Future Extension Roadmap

1. Smart OCR Receipt Scanning

Integration with Computer Vision / LLM multimodal APIs to automatically parse physical supermarket and restaurant receipts into itemized participant splits.

2. Multi-Currency International Trips

Real-time foreign exchange rate fetching with historical rate pinning so international travel expenses can be settled in local or home currencies.

3. Automated Bank Webhook Reconciliation

Direct payment gateway webhooks (e.g. Razorpay / UPI intent) to automatically transition settlements from PENDING to CONFIRMED upon bank settlement callbacks.

4. Export & Financial Accounting Reports

One-click CSV, Excel, and structured PDF financial statement generation for group trips, house rentals, and audit reconciliations.
